Python

- An object oriented interpreter-based programming language
- Basic essentials
  - Data types are numbers, strings, lists, and dictionaries (hash-tables)
  - For loops and conditionals
  - Functions
  - Lists and hash-tables are references (like pointers in C)
  - All variables are passed by value

Python Libraries for Data Science

Many popular Python toolboxes/libraries:

- NumPy
- SciPy
- Pandas
- SciKit-Learn

All these libraries are installed on the SCC

Visualization libraries

- matplotlib
- Seaborn

and many more …

Python Libraries for Data Science

NumPy:

- introduces objects for multidimensional arrays and matrices, as well as functions that allow to easily perform advanced mathematical and statistical operations on those objects

- provides vectorization of mathematical operations on arrays and matrices which significantly improves the performance

- many other python libraries are built on NumPy

Link: http://www.numpy.org/

Python Libraries for Data Science

SciPy:

- collection of algorithms for linear algebra, differential equations, numerical integration, optimization, statistics and more

- part of SciPy Stack

- built on NumPy

Link: https://www.scipy.org/scipylib/

Python Libraries for Data Science

Pandas:

- adds data structures and tools designed to work with table-like data (similar to Series and Data Frames in R)

- provides tools for data manipulation: reshaping, merging, sorting, slicing, aggregation etc.

- allows handling missing data

Link: http://pandas.pydata.org/

Python Libraries for Data Science

SciKit-Learn:

- an open source data analysis library, and the gold standard for Machine Learning (ML) in the Python ecosystem.
- provides machine learning algorithms: classification, regression, clustering, model validation etc.
- built on NumPy, SciPy and matplotlib

Link: http://scikit-learn.org/

Python Libraries for Data Science

matplotlib:

- python 2D plotting library which produces publication quality figures in a variety of hardcopy formats
- a set of functionalities similar to those of MATLAB
- line plots, scatter plots, barcharts, histograms, pie charts etc.
- relatively low-level; some effort needed to create advanced visualization

Link: https://matplotlib.org/

Python Libraries for Data Science

Seaborn:

- based on matplotlib
- provides high level interface for drawing attractive statistical graphics
- Similar (in style) to the popular ggplot2 library in R

Link: https://seaborn.pydata.org/

Introduction to Pandas' DataFrame

A Library that is Used for Data Manipulation and Analysis Tool Using Powerful Data Structures

Pandas First Steps: install and import

- Pandas is an easy package to install. Open up your terminal program (shell or cmd) and install it using either of the following commands:

```
$ conda install pandas
OR
$ pip  install  pandas
```

- For **`jupyter notebook`** users, you can run this cell:
  - The **!** at the beginning runs cells as if they were in a terminal.

```
!pip install pandas
```

- To import pandas we usually import it with a shorter name since it's used so much:

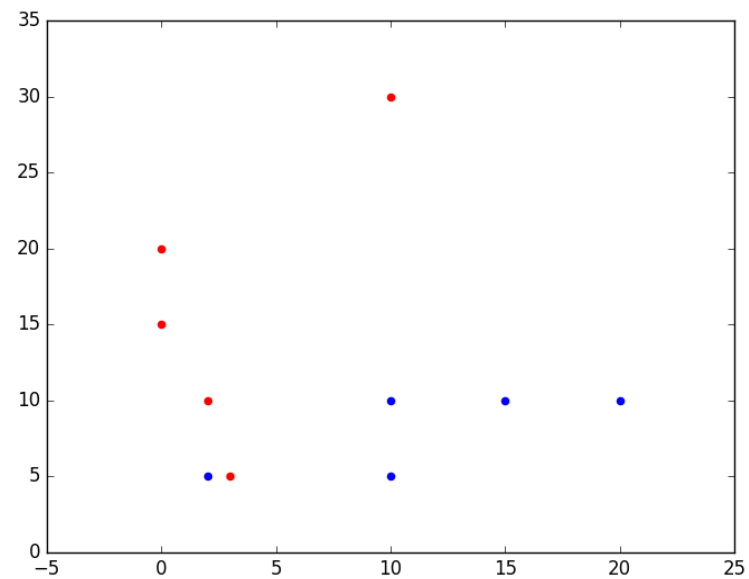
```
import pandas as pd
```

Installation: https://pandas.pydata.org/pandas-docs/stable/getting_started/install.html

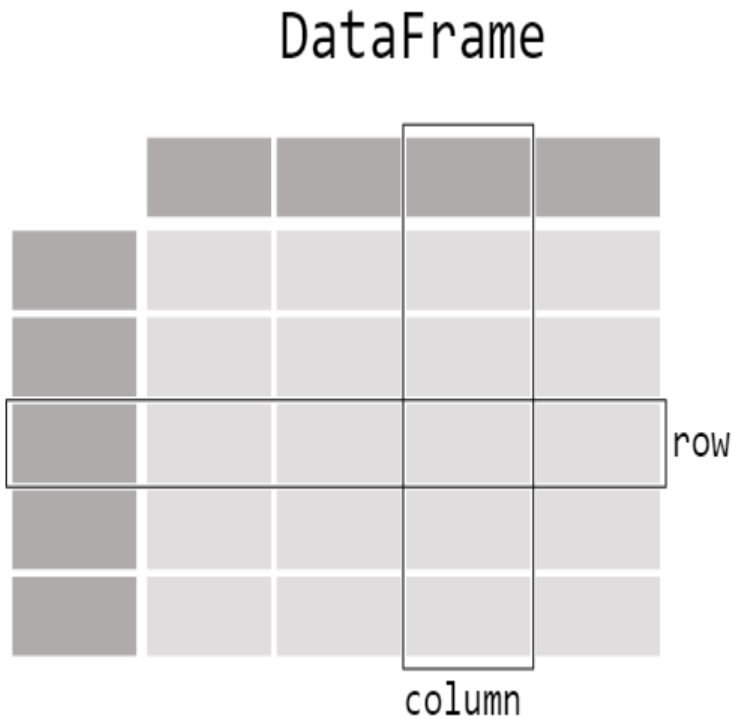
Data

- We think of data as vectors in a fixed dimensional space
- For example

Heart disease	Cigarettes per day	Exercise per day (mins)
1	10	10
1	2	5
1	20	10
1	10	5
1	15	10
0	10	30
0	2	10
0	3	5
0	0	20
0	0	15

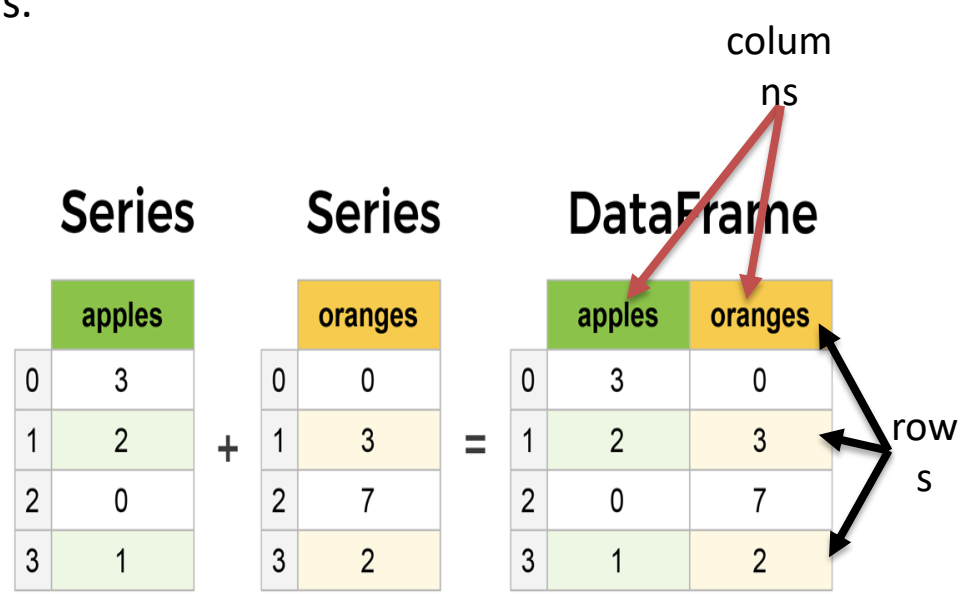


pandas: Data Table Representation



Core components of pandas: Series & DataFrameS

- The primary two components of pandas are the Series and DataFrame.
  - Series is essentially a column, and
  - DataFrame is a multi-dimensional table made up of a collection of Series.
- DataFrames and Series are quite similar in that many operations that you can do with one you can do with the other, such as filling in null values and calculating the mean.
  - A Data frame is a two-dimensional data structure, i.e., data is aligned in a tabular fashion in rows and columns.
- Features of DataFrame
  - Potentially columns are of different types
  - Size – Mutable
  - Labeled axes (*rows* and *columns*)
  - Can Perform Arithmetic operations on rows and columns



Types of Data Structure in Pandas

Data Structure	Dimensions	Description
Series	1	1D labeled homogeneous array with immutable size
Data Frames	2	General 2D labeled, size mutable tabular structure with potentially heterogeneously typed columns.
Panel	3	General 3D labeled, size mutable array.

- **Series & DataFrame**
  - Series is a one-dimensional array (1D Array) like structure with homogeneous data.
  - DataFrame is a two-dimensional array (2D Array) with heterogeneous data.
- **Panel**
  - Panel is a three-dimensional data structure (3D Array) with heterogeneous data.
  - It is hard to represent the panel in graphical representation.
  - But a panel can be illustrated as a container of DataFrame

pandas.DataFrame

```
pandas.DataFrame(data, index, columns,
dtype, copy )
```

- data: data takes various forms like *ndarray*, *series*, *map*, *lists*, *dict*, constants and also another *DataFrame*.
- index: For the **row labels**, that are to be used for the resulting frame, Optional, Default is *np.arange(n)* if no index is passed.
- columns: For **column labels**, the optional default syntax is - *np.arange(n)*. This is only true if no index is passed.
- dtype: Data type of each column.
- copy: This command (or whatever it is) is used for copying of data, if the default is False.

- **Create DataFrame**
  - A pandas DataFrame can be created using various inputs like –
    - Lists
    - dict
    - Series
    - Numpy ndarrays
    - Another DataFrame

Creating a DataFrame from scratch

- There are many ways to create a DataFrame from scratch, but a great option is to just use a simple dict. But first you must import pandas.

```
import pandas as pd
```

- Let's say we have a fruit stand that sells apples and oranges. We want to have a column for each fruit and a row for each customer purchase. To organize this as a dictionary for pandas we could do something like:

```
data = { 'apples':[3, 2, 0, 1] , 'oranges':[0
, 3, 7, 2] }
```

- And then pass it to the pandas DataFrame constructor:

```
df = pd.DataFrame(data)
```

	apples	oranges
0	3	0
1	2	3
2	0	7
3	1	2

How did that work?

- Each (key, value) item in data corresponds to a column in the resulting DataFrame.
- The Index of this DataFrame was given to us on creation as the numbers **0–3**, but we could also create our own when we initialize the DataFrame.
- E.g. if you want to have customer names as the index:

```
df = pd.DataFrame(data, index=['Ahmad', 'Ali', 'Rashed'
, 'Hamza'])
```

	apples	oranges
Ahmad	3	0
Ali	2	3
Rashed	0	7
Hamza	1	2

- So now we could locate a customer's order by using their names:

```
df.loc['Ali']
```

```
apples     2
oranges    3
Name: Ali, dtype: int64
```

pandas.DataFrame.from_dict

```
pandas.DataFrame.from_dict(data, orient='columns',
dtype=None, columns=None)
```

- **data** : dict
  - Of the form **`{field:array-like}`** or **`{field:dict}`**.

- **orient** : **`{'columns', 'index'}`**, default **`'columns'`**
  - The "orientation" of the data.
  - If the keys of the passed dict should be the columns of the resulting DataFrame, pass 'columns' (default).
  - Otherwise if the keys should be rows, pass 'index'.

- **dtype** : **`dtype`**, default **`None`**
  - Data type to force, otherwise infer.

- **columns** : **`list`**, default **`None`**
  - Column labels to use when **`orient='index'`**. Raises a **`ValueError`** if used with **`orient='columns'`**.

https://pandas.pydata.org/pandas-docs/version/0.23/generated/pandas.DataFrame.from_dict.html

pandas' **orient** keyword

```
data = {'col_1':[3, 2, 1, 0], 'col_2':['a'
,'b','c','d']}
pd.DataFrame.from_dict(data)
```

	col_1	col_2
0	3	a
1	2	b
2	1	c
3	0	d

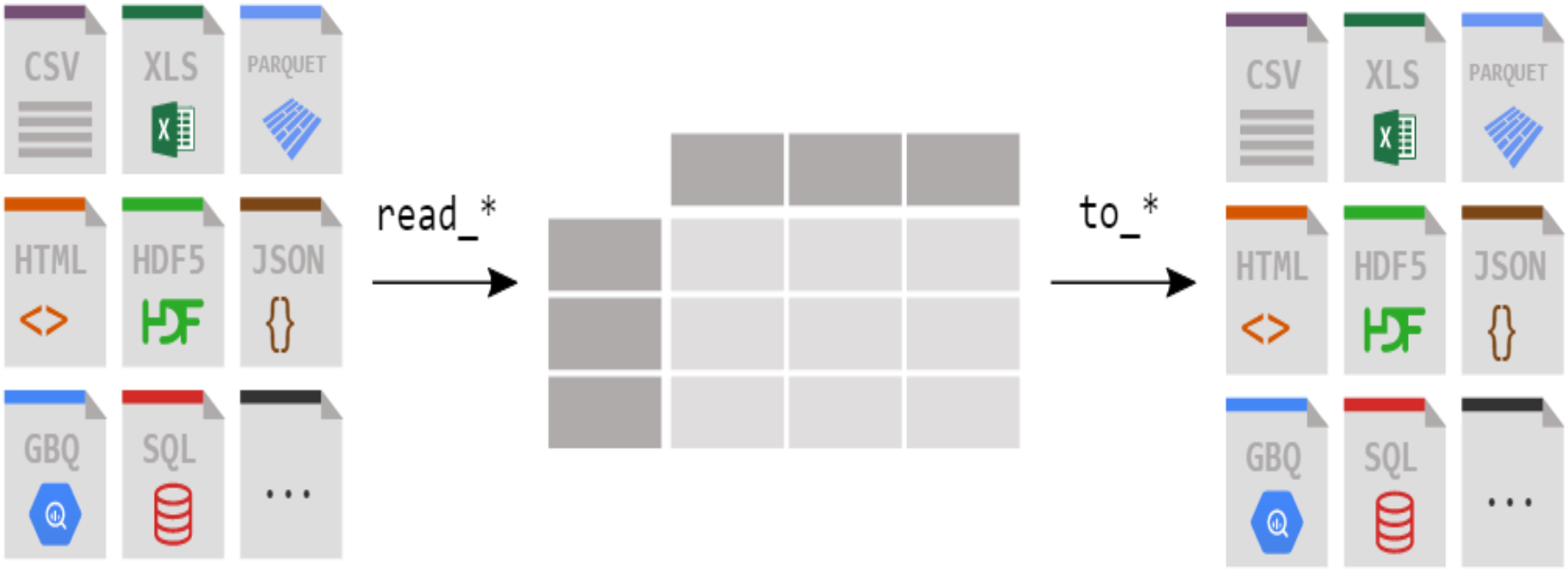
```
data = {'row_1':[3, 2, 1, 0], 'row_2':['a'
,'b','c','d']}
pd.DataFrame.from_dict(data,
                       orient='index')
```

	0	1	2	3
row_1	3	2	1	0
row_2	a	b	c	d

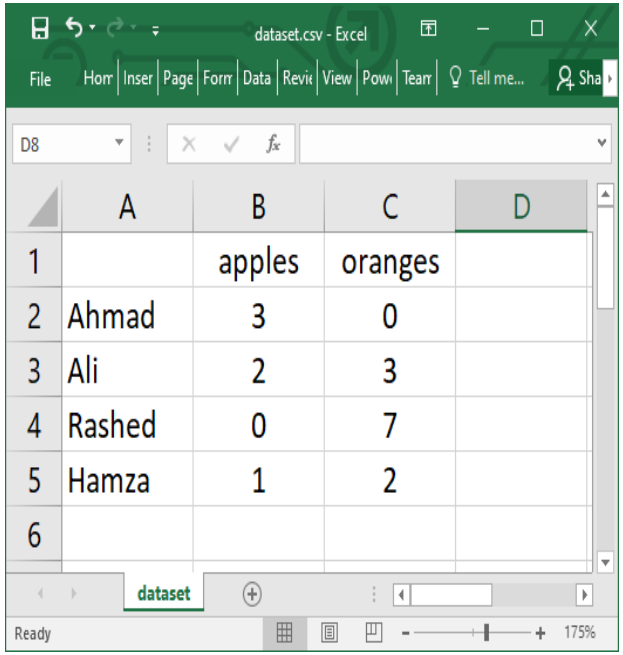
```
data = {'row_1':[3, 2, 1, 0], 'row_2':['a'
,'b','c','d']}
pd.DataFrame.from_dict(data,
                       orient  = 'index',
                       columns =
['A','B','C','D'])
```

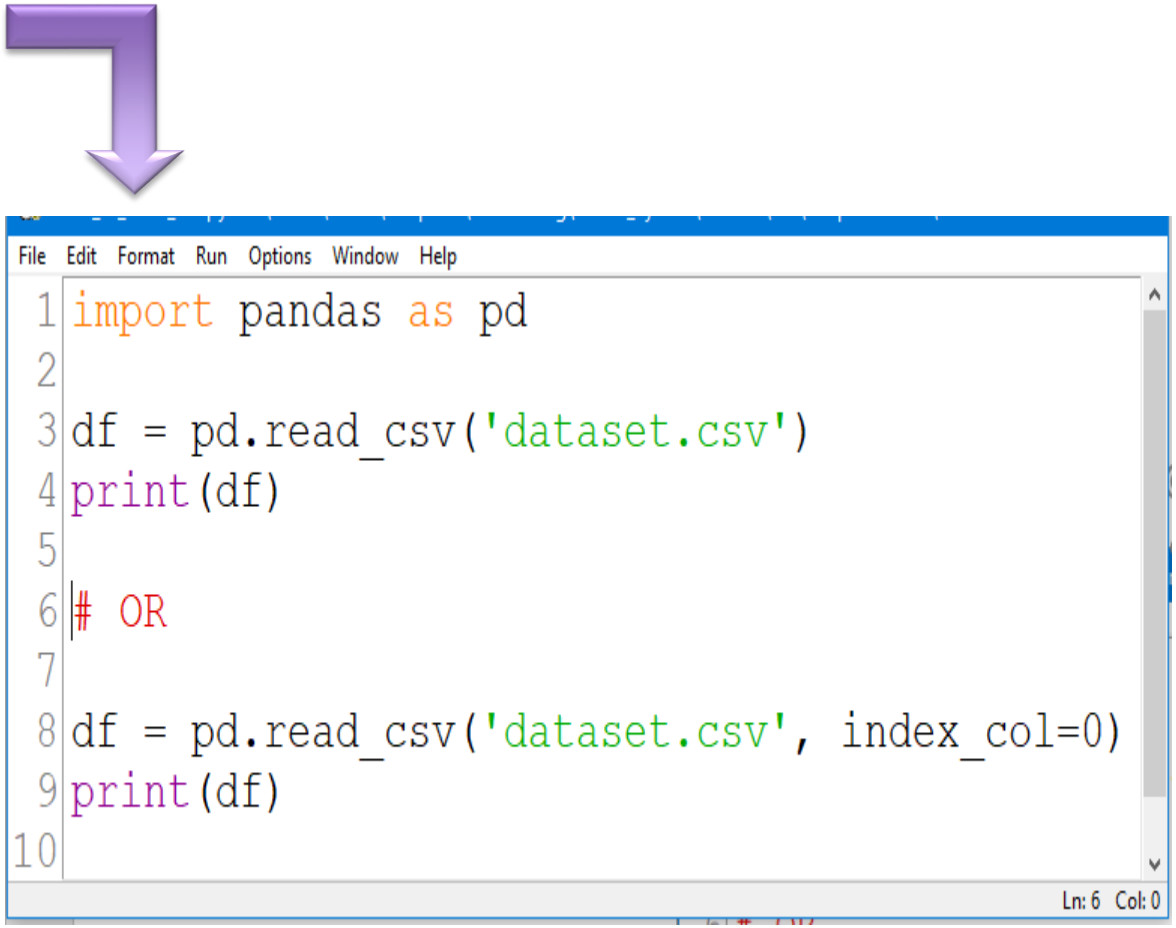
	A	B	C	D
row_1	3	2	1	0
row_2	a	b	c	d

LOADING A DATAFRAME FROM FILES



Reading data from a CSV file





Reading data from CSVs

- With CSV files, all you need is a single line to load in the data:

```
df = pd.read_csv('dataset.csv')
```

	Unnamed: 0	apples	oranges
0	Ahmad	3	0
1	Ali	2	3
2	Rashed	0	7
3	Hamza	1	2

- CSVs don't have indexes like our DataFrames, so all we need to do is just designate the **`index_col`** when reading:

```
df = pd.read_csv('dataset.csv', index_col=0)
```

	apples	oranges
Ahmad	3	0
Ali	2	3
Rashed	0	7
Hamza	1	2

- *Note: here we're setting the <u>index to be column zero</u>.*

Reading data from JSON

- If you have a JSON file — which is essentially a stored Python dict — pandas can read this just as easily:

```
df = pd.read_json('dataset.json')
```

- Notice this time our index came with us correctly since using JSON allowed indexes to work through nesting.
- Pandas will try to figure out how to create a DataFrame by analyzing structure of your JSON, and sometimes it doesn't get it right.
- Often you'll need to set the `orient` keyword argument depending on the structure

Example #1:Reading data from JSON

```
{
  "apples" :{"Ahmad":3,"Ali":2,"Rashed":0, "Hamza":1},
  "oranges":{"Ahmad":0,"Ali":3,"Rashed":7, "Hamza":2}
}
```

File Edit Format Run Options Window Help

```
import pandas as pd

df = pd.read_json('dataset.json')
print(df)
```

Ln: 1 Col: 0

	apples	oranges
Ahmad	3	0
Ali	2	3
Rashed	0	7
Hamza	1	2

Example #2: Reading data from JSON

```
{
  "Ahmad"  : {"apples":3,"oranges":0},
  "Ali"    : {"apples":2,"oranges":3},
  "Rashed" : {"apples":0,"oranges":7},
  "Hamza"  : {"apples":1,"oranges":2}
}
```

File Edit Format Run Options Window Help

```
import pandas as pd

df = pd.read_json('dataset.json')
print(df)
```

Ln: 1 Col: 0

	Ahmad	Ali	Rashed	Hamza
apples	3	2	0	1
oranges	0	3	7	2

Example #3: Reading data from JSON

```
{
  "Ahmad"  : {"apples":3,"oranges":0},
  "Ali"    : {"apples":2,"oranges":3},
  "Rashed" : {"apples":0,"oranges":7},
  "Hamza"  : {"apples":1,"oranges":2}
}
```

```
import pandas as pd

df = pd.read_json('dataset.json',
                  orient='column')
print(df)
```

	Ahmad	Ali	Rashed	Hamza
apples	3	2	0	1
oranges	0	3	7	2

```
import pandas as pd

df = pd.read_json('dataset.json',
                  orient='index')
print(df)
```

	apples	oranges
Ahmad	3	0
Ali	2	3
Rashed	0	7
Hamza	1	2

Converting back to a CSV or JSON

- So after extensive work on cleaning your data, you're now ready to save it as a file of your choice. Similar to the ways we read in data, pandas provides intuitive commands to save it:

```
df.to_csv('new_dataset.csv')
df.to_json('new_dataset.json
')
```

- When we save JSON and CSV files, all we have to input into those functions is our desired filename with the appropriate file extension.

Reference: https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.to_json.html

Start Jupyter nootebook

On the Shared Computing Cluster

```
[scc1 ~] jupyter notebook
```

Loading Python Libraries

```
In [ ]:  #Import Python Libraries
         import numpy as np
         import scipy as sp
         import pandas as pd
         import matplotlib as mpl
         import seaborn as sns
```

Press `Shift+Enter` to execute the *jupyter* cell

Reading data using pandas

```
In [ ]:#Read csv file
       df =
       pd.read_csv("http://rcs.bu.edu/examples/python/data_analysis/
       Salaries.csv")
```

Note: The above command has many optional arguments to fine-tune the data import process.

There is a number of pandas commands to read other data formats:

```
pd.read_excel('myfile.xlsx',sheet_name='Sheet1',
index_col=None, na_values=['NA'])
pd.read_stata('myfile.dta')
pd.read_sas('myfile.sas7bdat')
pd.read_hdf('myfile.h5','df')
```

Exploring data frames

In [3]:
```
#List first 5 records
df.head()
```

Out[3]

	rank	discipline	phd	service	sex	salary
0	Prof	B	56	49	Male	186960
1	Prof	A	12	6	Male	93000
2	Prof	A	23	20	Male	110515
3	Prof	A	40	31	Male	131205
4	Prof	B	20	18	Male	104800

Hands-on exercises

- ✓ Try to read the first 10, 20, 50 records;
- ✓ Can you guess how to view the last few records

Hint:

Data Frame data types

Pandas Type	Native Python Type	Description
object	string	The most general dtype. Will be assigned to your column if column has mixed types (numbers and strings).
int64	int	Numeric characters. 64 refers to the memory allocated to hold this character.
float64	float	Numeric characters with decimals. If a column contains numbers and NaNs(see below), pandas will default to float64, in case your missing value has a decimal.
datetime64, timedelta[ns]	N/A (but see the datetime module in	Values meant to hold time data. Look into these for

Data Frame data types

In [4]:
```
#Check a particular column type
df['salary'].dtype
```

Out[4]: dtype('int64')

In [5]:
```
#Check types for all the columns
df.dtypes
```

Out[4]:

rank	object
discipline	object
phd	int64
service	int64
sex	object
salary	int64

dtype: object

Data Frames attributes

Python objects have *attributes* and *methods*.

df.attribute	description
dtypes	list the types of the columns
columns	list the column names
axes	list the row labels and column names
ndim	number of dimensions
size	number of elements
shape	return a tuple representing the dimensionality
values	numpy representation of the data

Hands-on exercises

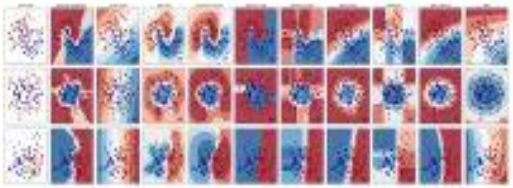
- ✓ Find how many records this data frame has;
- ✓ How many elements are there?
- ✓ What are the column names?
- ✓ What types of columns we have in this data frame?

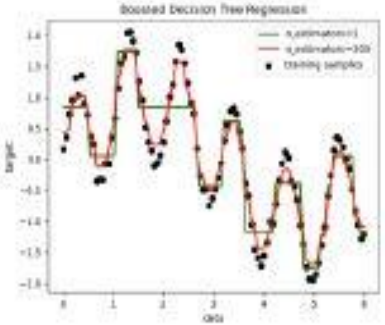
What is scikit-learn?

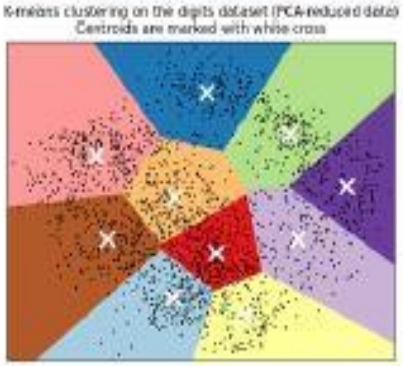
- Simple and efficient tools for predictive data analysis
  - Machine Learning methods
  - Data processing
  - Visualization
- Accessible to everybody, and reusable in various contexts
  - Documented API with lot's of examples
  - Not bound to Training frameworks (e.g. Tensorflow, Pytorch)
  - Building blocks for your data analysis
- Built on *NumPy*, *SciPy*, and *matplotlib*
  - No own data types (unlike Pandas)
  - Benefit from NumPy and SciPy optimizations
  - Extends the most common visualisation tool
- Open source, commercially usable - BSD license

Tools of scikit-learn

- **Classification:**
  Categorizing objects to one or more classes.
  - Support Vector Machines (SVM)
  - Nearest Neighbors
  - Random Forest
  - ...
- **Regression:**
  Prediction of one (uni-) or more (multi-variate) continuous-valued attributes.
  - Support Vector Regression (SVR)
  - Nearest Neighbors
  - Random Forest
  - ...
- **Clustering:**
  Group objects of a set.
  - k-Means
  - Spectral Clustering

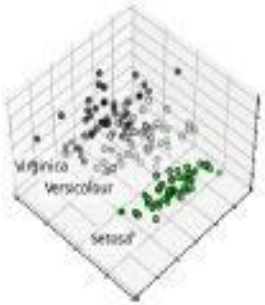




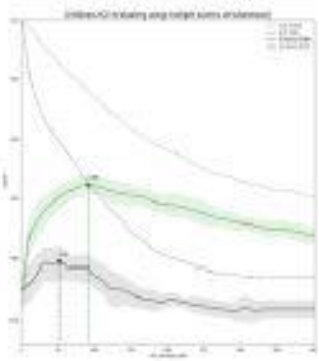


Tools of scikit-learn - cont'd

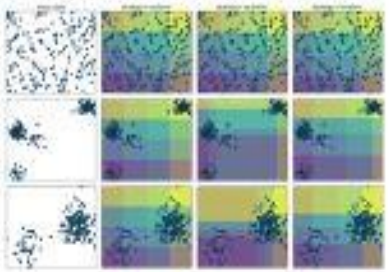
- **Dimensionality reduction:**
  Reducing the number of random variables.
  - Principal Component Analysis (PCA)
  - Feature Selection
  - non-Negative Matrix Factorization
  - ...



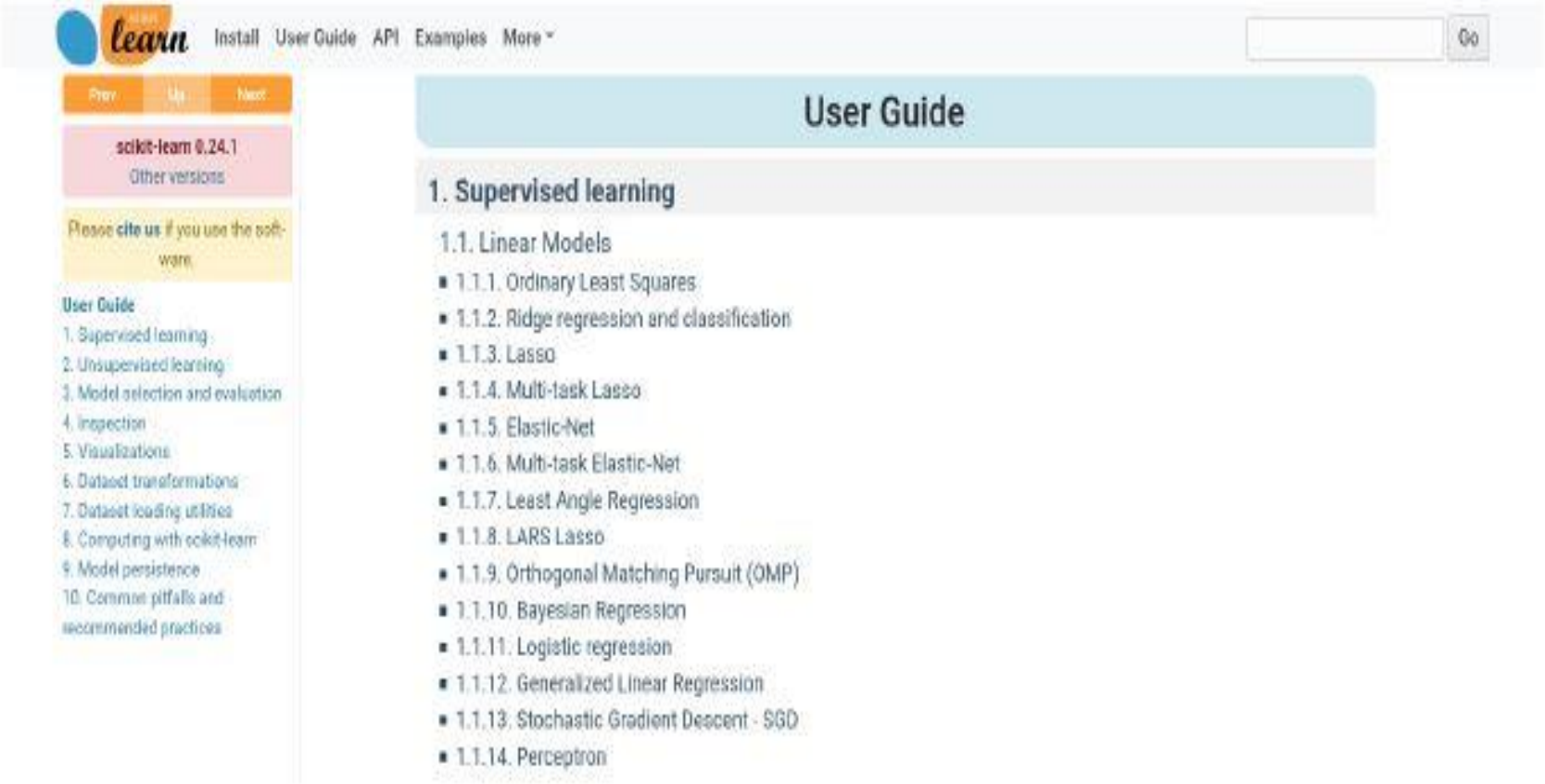
- **Model selection:**
  Compare, validate and choose parameters/models.
  - Grid Search
  - Cross Validation
  - ...



- **Pre-Processing:**
  Prepare/transform data before training models.
  - Conversion
  - Normalization
  - Feature Extraction
  - ...

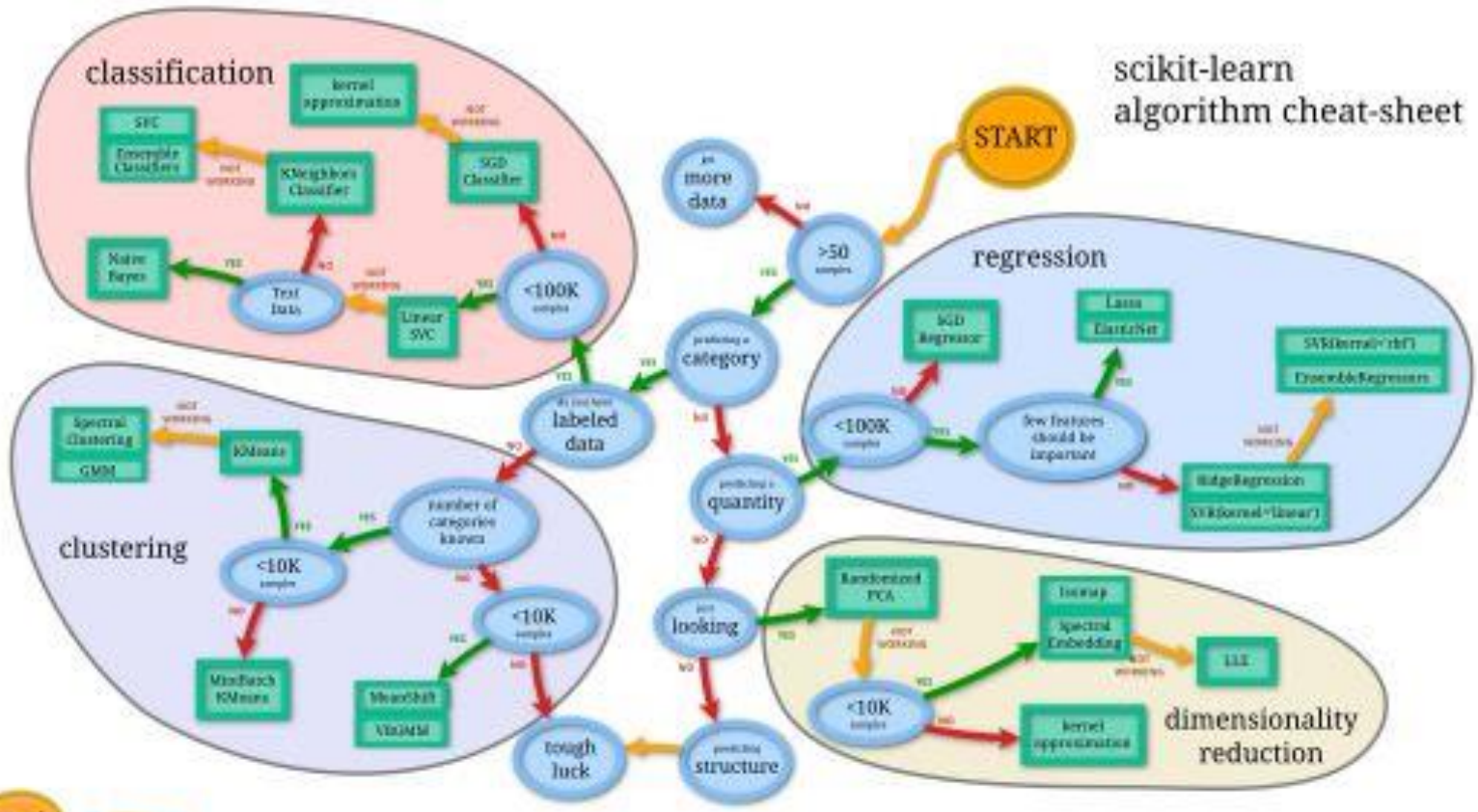


User Guide



The user guide can be found here

Choosing the Right Estimator



Linked map can be found [here]

How to Get scikit-learn

- Open Source (BSD License) available on Github
- Current version: *0.24.2*
- Easy install via PIP or Conda for Windows, macOS and Linux, e.g:

```
$ pip install scikit-learn or
$ conda install -c intel scikit-learn
```

Programming Model

- Builds on *NumPy*, *SciPy* and *matplotlib*:
  - Avoids conversion of data types
  - Can be integrated seamlessly, even with Tensorflow and Pytorch
  - Benefits from performance optimizations of BLAS, FFT, etc. optimizations
- scikit-learn available as Python module:

```
import sklearn
sklearn.show_versions()

System:
    python: 3.8.6 | packaged by conda-forge | (default, Dec 26 2020, 05:05:16)  [GCC 9.3.0]
executable: /opt/conda/bin/python
   machine: Linux-3.10.0-1127.13.1.el7.x86_64-x86_64-with-glibc2.10

Python dependencies:
          pip: 20.3.3
   setuptools: 49.6.0.post20201009
      sklearn: 0.24.0
        numpy: 1.19.5
        scipy: 1.5.3
       Cython: 0.29.21
       pandas: 1.1.5
   matplotlib: 3.3.3
       joblib: 1.0.0
threadpoolctl: 2.1.0

Built with OpenMP: True
```

- Typical input (`n_samples`, `n_features`), but others are also possible

Example Datasets

- Easy access to toy datasets via sklearn.datasets:

Boston house prices dataset

Iris plants dataset

Diabetes dataset

Optical recognition of handwritten digits dataset

Linnerrud dataset

Wine recognition dataset

Breast cancer wisconsin (diagnostic) dataset

Loading via:

Function	Description
`load_boston(*[, return_X_y])`	Load and return the boston house-prices dataset (regression).
`load_iris(*[, return_X_y, as_frame])`	Load and return the iris dataset (classification).
`load_diabetes(*[, return_X_y, as_frame])`	Load and return the diabetes dataset (regression).
`load_digits(*[, n_class, return_X_y, as_frame])`	Load and return the digits dataset (classification).
`load_linnerud(*[, return_X_y, as_frame])`	Load and return the physical excercise linnerud dataset.
`load_wine(*[, return_X_y, as_frame])`	Load and return the wine dataset (classification).
`load_breast_cancer(*[, return_X_y, as_frame])`	Load and return the breast cancer wisconsin dataset (classification).

Using Example Datasets – cont'd

- Convention:
  - X: Data for training/prediction
  - y: Label in case of supervised learning (aka. target)
  - n_class: How many classes from the set should be retrieved
  - return_X_y: Boolean whether tuple of data and label is desired
  - as_frame: Boolean whether data should be a Pandas DataFrame
- Example:

```
import sklearn.datasets

sk_digits = sklearn.datasets.load_digits(n_class=2,
                                         return_X_y=True,
                                         as_frame=False)
print(sk_digits)

(array([[ 0.,  0.,  5., ...,  0.,  0.,  0.],
        ...,
        [ 0.,  0.,  6., ...,  6.,  0.,  0.]]),
 array([0, 1, 0, 1, 0, 1, 0, 0,
        ...,
        1, 1, 1, 1, 1, 0, 1, 0]))
```

Using Example Datasets

Returns:

data : *Bunch*

Dictionary-like object, with the following attributes.

data : *{ndarray, dataframe} of shape (1797, 64)*

The flattened data matrix. If `as_frame=True`, `data` will be a pandas DataFrame.

target: {ndarray, Series} of shape (1797,)

The classification target. If `as_frame=True`, `target` will be a pandas Series.

feature_names: list

The names of the dataset columns.

target_names: list

The names of target classes.

New in version 0.20.

frame: DataFrame of shape (1797, 65)

Only present when `as_frame=True`. DataFrame with `data` and `target`.

New in version 0.23.

images: {ndarray} of shape (1797, 8, 8)

The raw image data.

DESCR: str

The full description of the dataset.

(data, target) : *tuple if `return_X_y` is True*

New in version 0.18.

This is a copy of the test set of the UCI ML hand-written digits datasets

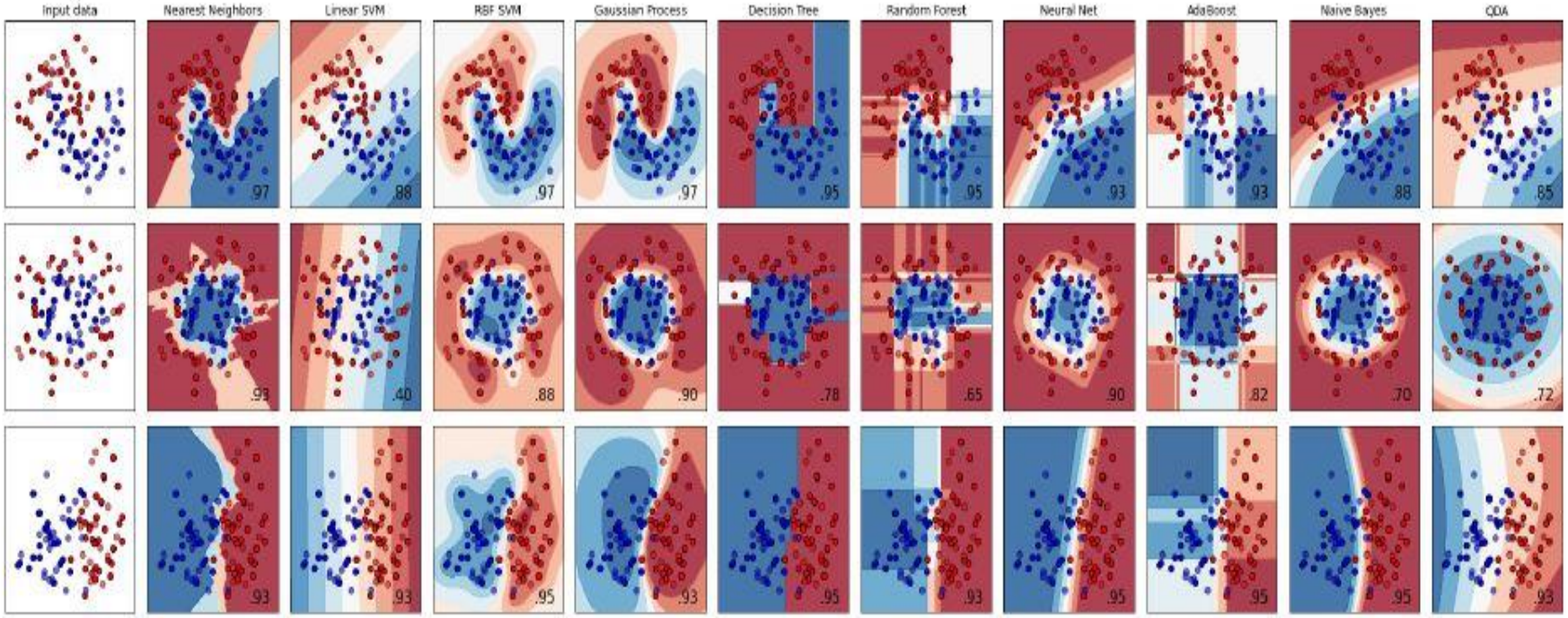
https://archive.ics.uci.edu/ml/datasets/Optical+Recognition+of+Handwritten+Digits

Classification – Informally…

- Widely used task: given data determine the class it belongs to. The class will lead to a decision or outcome.
- Used in many different places:
  - DNA and protein sequence classification
  - Insurance
  - Weather
  - Experimental physics: Higgs Boson determination

Classification - formally

- Supervised:
  Label information is available and can be used for learning
- Unsupervised:
  No (initial) labels and learning needs to structure data on its own
- Many classification methods exist:



From scikit-learn documentation: [refer here]

A classification Model

Regression in scikit-learn

- Classification vs. Regression[1]:

  Classify for categorical output

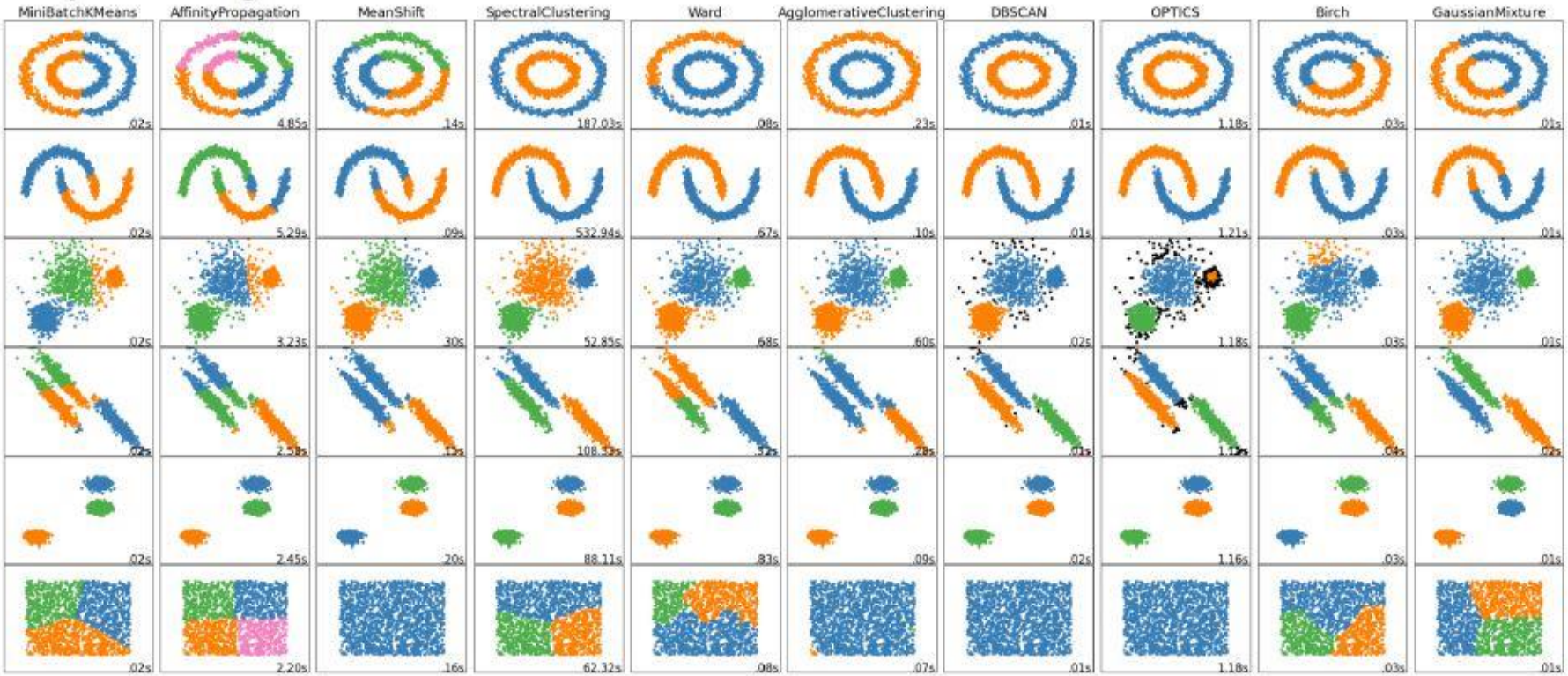
  Regression: predicting continuous-valued
- Can be "by-products" of classification methods, e.g.:

  RandomForestClassifier and RandomForestRegressor, or SVC and SVR

[1]As scikit-learn regards it.

Clustering

- Many clustering methods exist:



From scikit-learn documentation: [refer here]

Clustering

- Unsupervised: Find clusters (set of classes) automatically

  Clustering is applied in two steps:

  1. Train (i.e. identify) cluster with training data
  2. Retrieve the labels/metrics from the trained model

Table can be found in: refer here

Model Selection

- For Estimators:

  Cross-Validation

  Tuning Hyper-Parameters
- Metrics and Scoring
- Validation Curves

Pre-Processing

- Standardization, or mean removal and variance scaling
- Non-linear transformation (e.g. mapping to distributions)
- Normalization
- Encoding categorical features
- Discretization
- Imputation of missing values
- Generating polynomial features
- Custom transformers

What Method is the Best for Me?

We cannot answer that instantly, but consider the following requirements:

- How much training data do you have?
- Is your problem continuous or discrete?
- What is the ratio #*features* and #*samples*?
- Do you need a sparse model?
- Would reducing dimensionality be an option?
- Do you have a multi-task/-label problem?

Here's a great overview of (some) of the methods: Data Science Cheatsheet

Basic statistical Analysis

statsmodel and scikit-learn - both have a number of function for statistical analysis

The first one is mostly used for regular analysis using R style formulas, while scikit-learn is more tailored for Machine Learning.

statsmodels:

- linear regressions
- ANOVA tests
- hypothesis testings
- many more ...

scikit-learn:

- kmeans
- support vector machines
- random forests
- many more ...

Over to a demo in Jupyter…

Conclusions

- We saw basic data science and machine learning tasks in Python scikit-learn
- Can we handle very large datasets in Python scikit-learn? Yes
  - For space use array from numpy to use a byte for a char and 4 for float and int. Otherwise more space is used because Python is object oriented
  - For speed use stochastic gradient descent in scikit-learn (doesn’t come with mini-batch though) and mini-batch k-means
- Deep learning stuff: Keras

References

- pandas documentation
  - https://pandas.pydata.org/pandas-docs/stable/index.html
- pandas: Input/output
  - https://pandas.pydata.org/pandas-docs/stable/reference/io.html
- pandas: DataFrame
  - https://pandas.pydata.org/pandas-docs/stable/reference/frame.html
- pandas: Series
  - https://pandas.pydata.org/pandas-docs/stable/reference/series.html
- pandas: Plotting
  - https://pandas.pydata.org/pandas-docs/stable/reference/plotting.html
- Scikit-learn
  - https://web.njit.edu/~usman/courses/cs677_summer20/background.pptx
  - https://www.unmc.edu/bsbc/education/jclub/JournalClub_sklearn_Abrar_01-13-2020.pptx
  - https://events.prace-ri.eu/event/1333/attachments/1968/3903/1_Introduction_to_scikit-learn.pdf

Thank you

Questions?

Answers!

email: oswald@nitt.edu (C. Oswald)